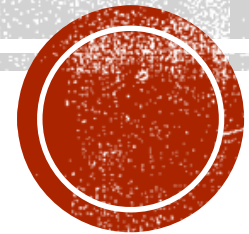


INTRODUCTION TO RECONFIGURABLE COMPUTING

CS G553: Reconfigurable computing

Dr. Meetha V Shenoy

Lect-6



ARCHITECTURE OF FPGA FABRICS

- Logic Elements
- Logic Interconnects
- I/O Blocks/Pins



ARCHITECTURE OF FPGA FABRICS

- How many logic elements are there in the FPGA?
- LE structure:
 - How many inputs?
 - Dedicated logic?
- What types of interconnect?
 - How many of each type?-How long are the interconnect segments?
 - Interconnects are laid out Uniform or non-uniform over chip?



FPGA Design Flow

CAD
Tools

Design Entry

Schematic editor, HDL, FSM
Editor

Functional
Simulation

Check correctness using test
patterns

Synthesis

Translate to Boolean functions and
Netlist formats → EDIF, XNF
then LUT based technology
mapping

Place and
Route

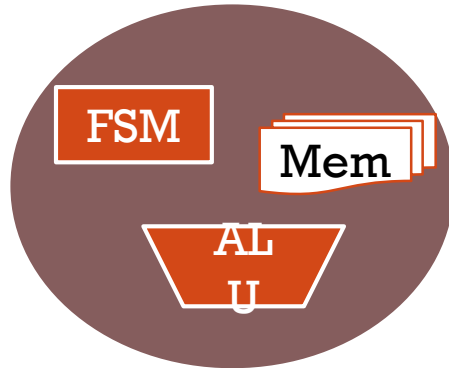
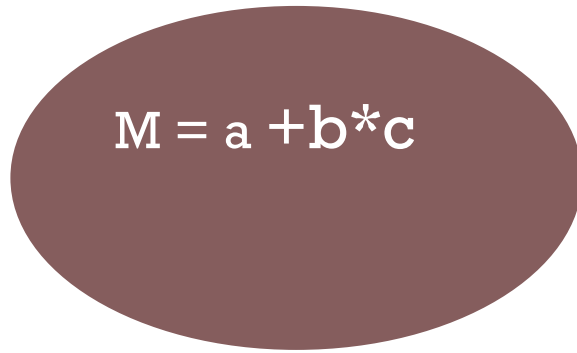
Operators should be placed and
connected

Configurati
on
Bitstream

Description of all the bits used to
configure LUTs, Interconnects, state of
Mux and I/O

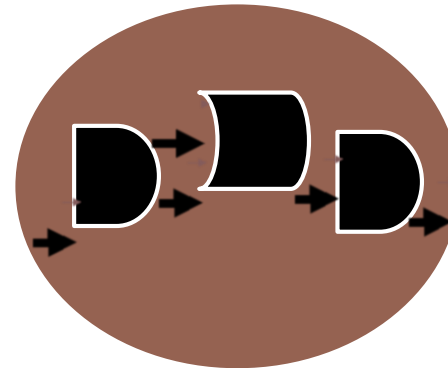
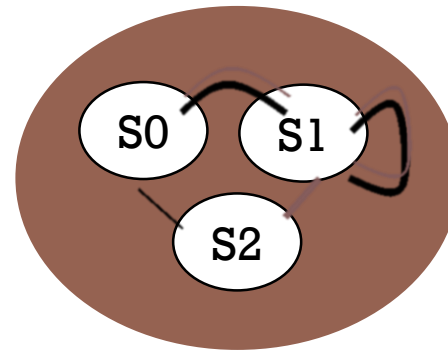


TAXONOMY OF SYNTHESIS

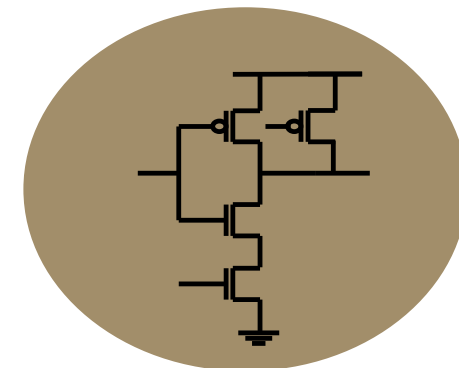
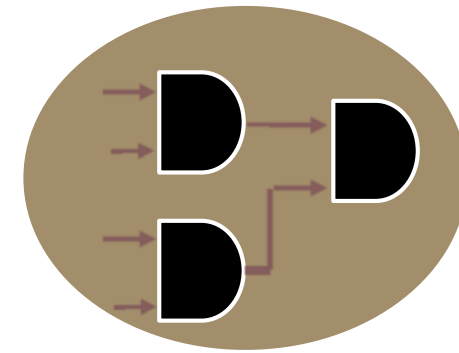


Architectural Synthesis

RC@Meetha V Shenoy



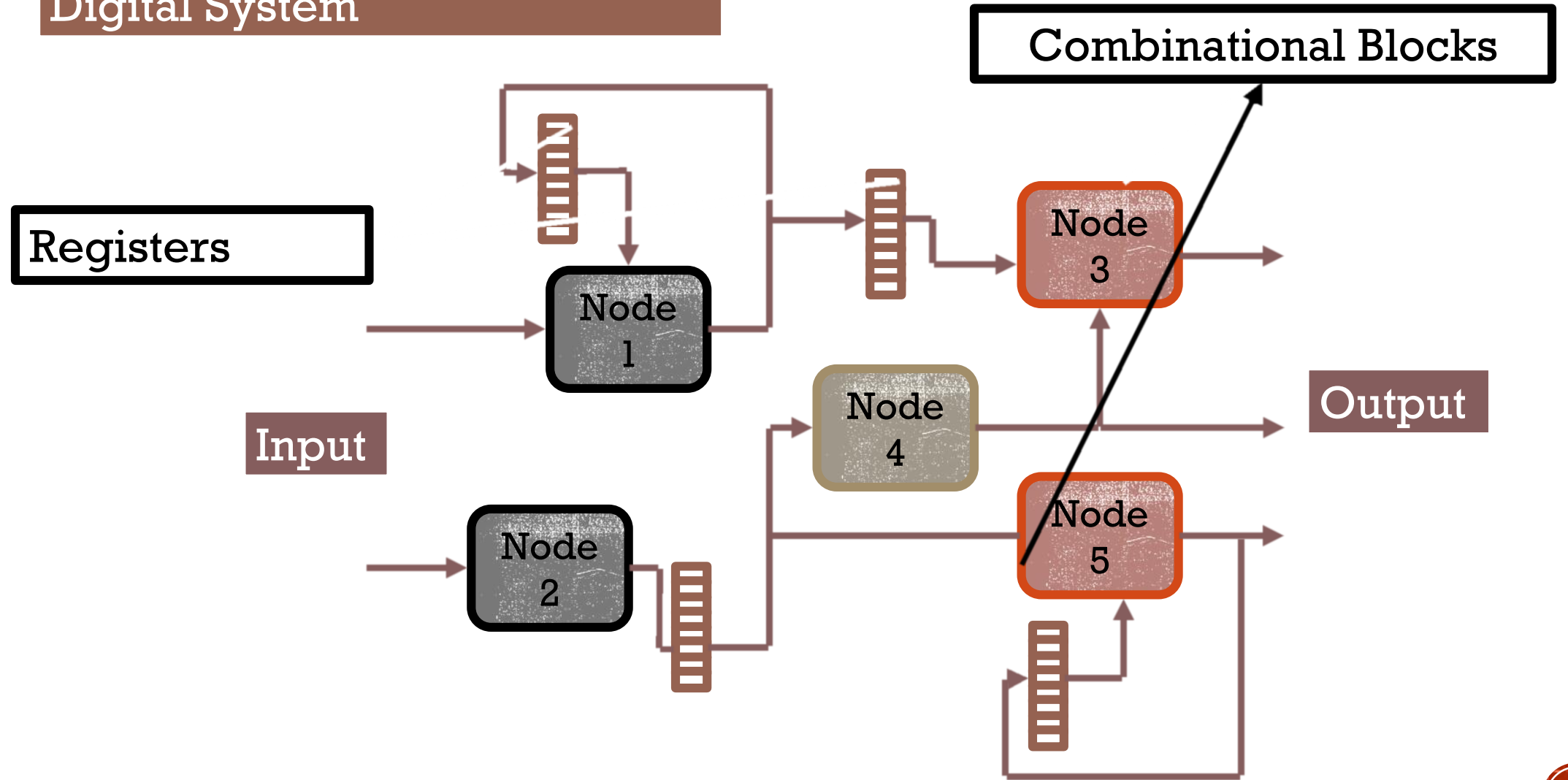
Logic Synthesis



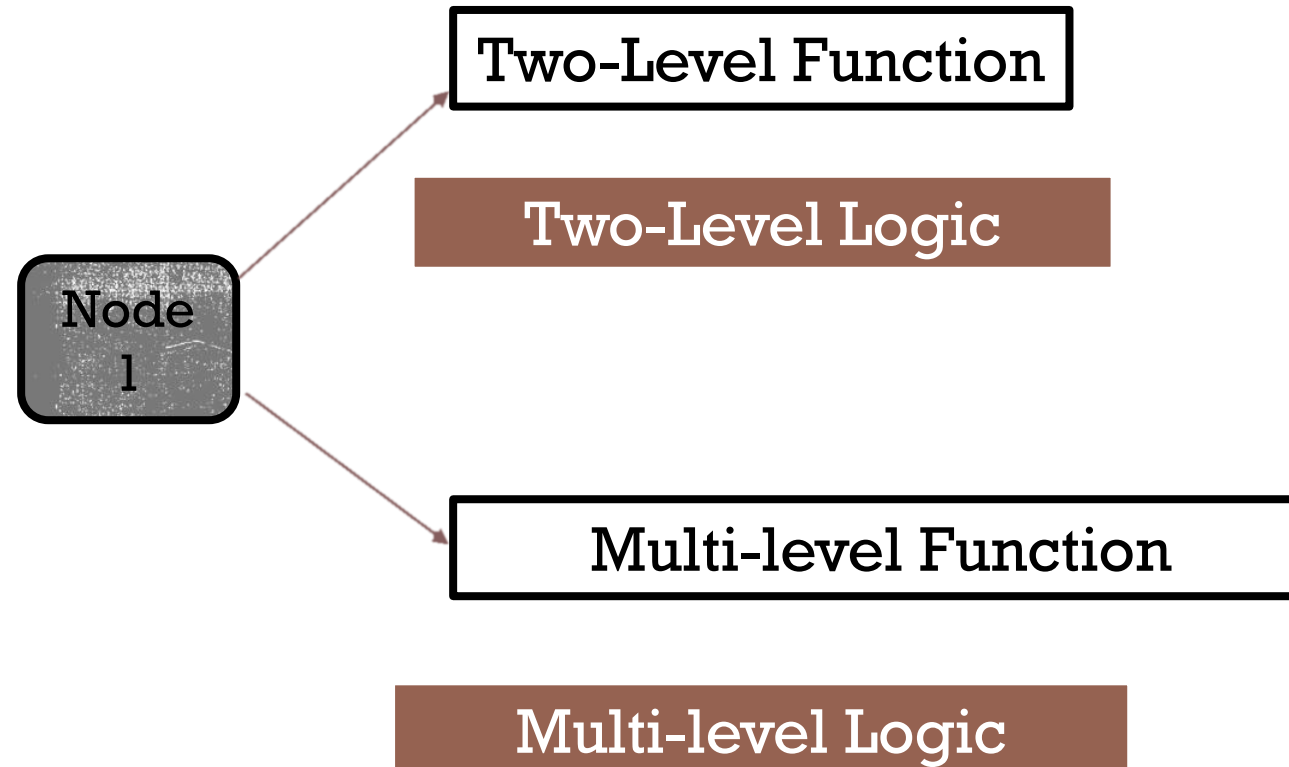
Circuit Synthesis



Digital System



LOGIC SYNTHESIS

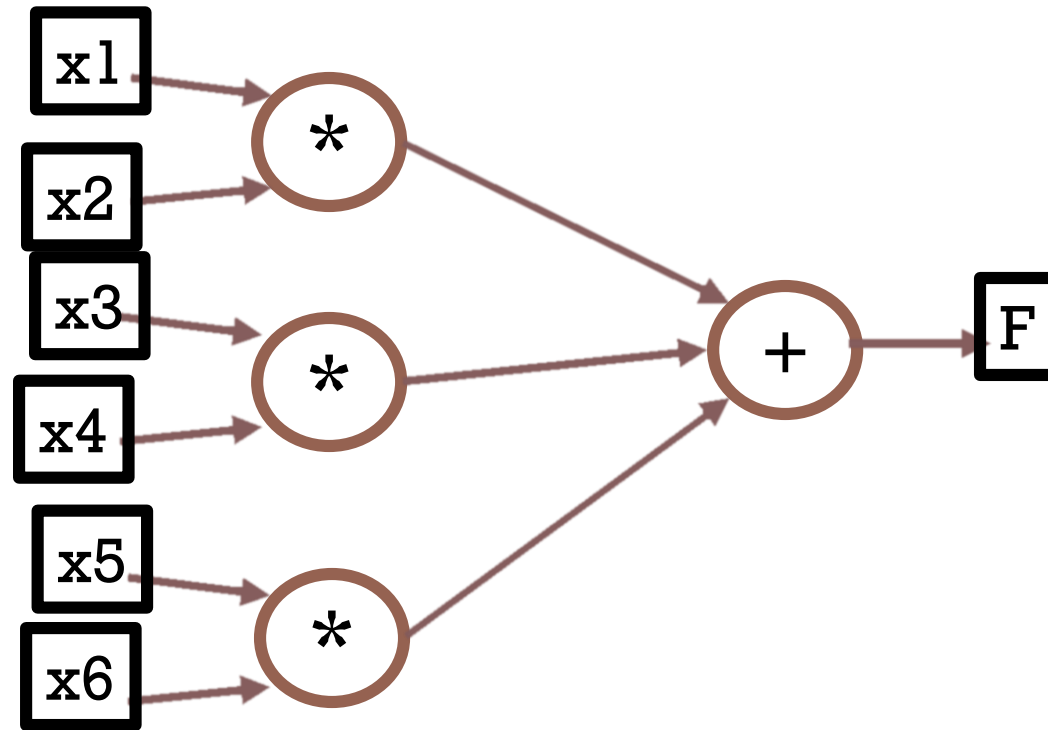


TWO LEVEL LOGIC

- Two-level synthesis deals with the synthesis of designs represented in two-level logic.
- Two level logic- number of gates crossed in the longest path is two
- Sum of Product and Product of Sum Representation
- Suitable for PLA &PAL implementation



TWO LEVEL LOGIC

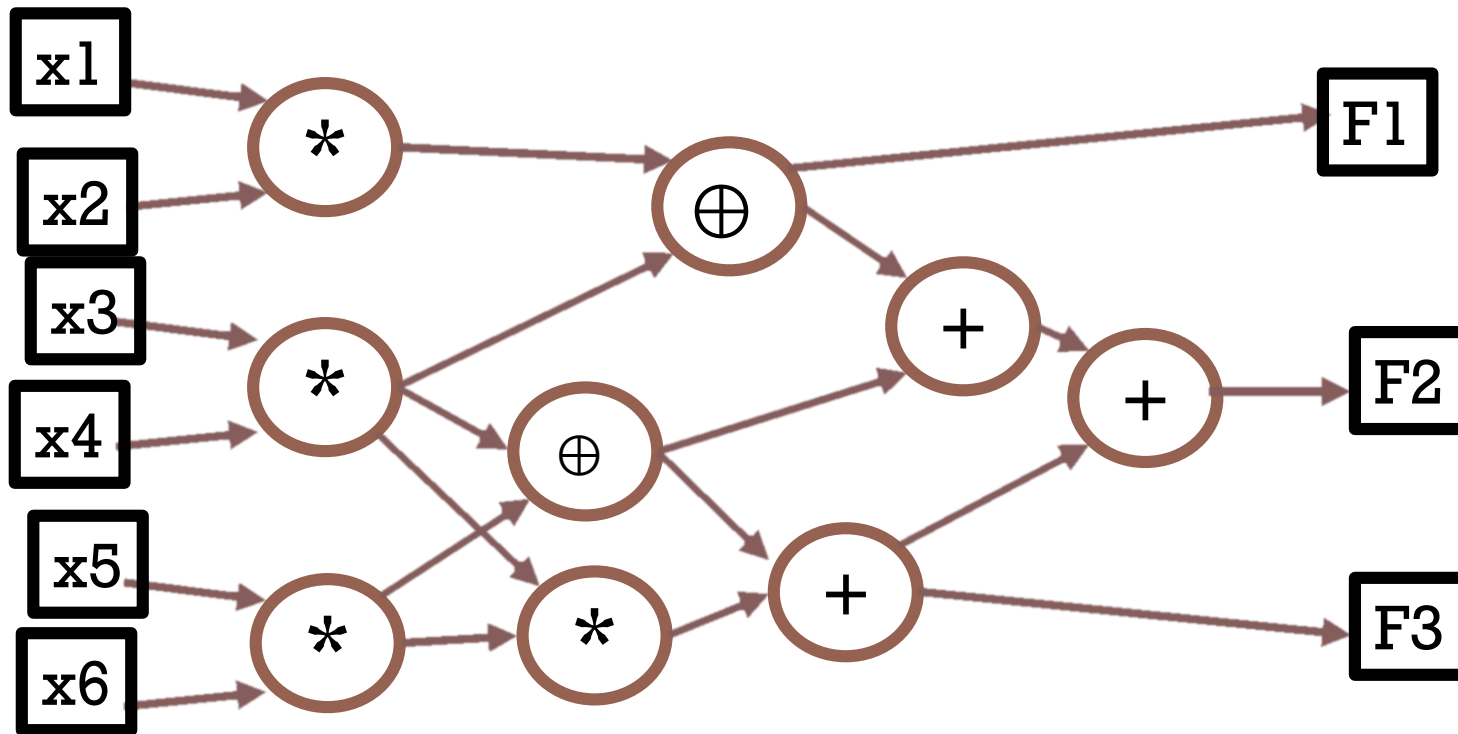


MULTI-LEVEL LOGIC SYNTHESIS

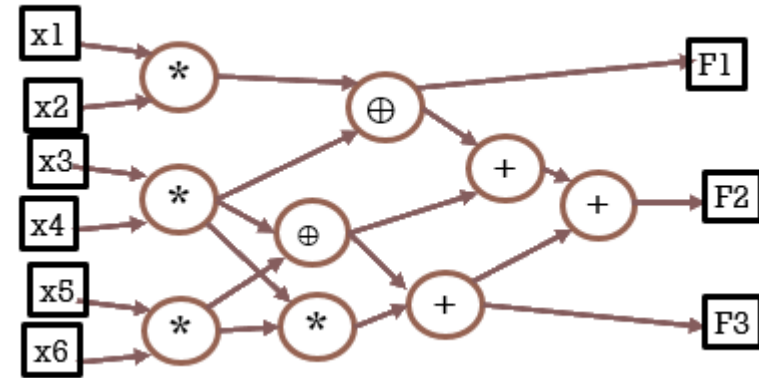
- The number of gates crossed in the longest path more than two
- Mostly used in standard cell and field programmable devices



MULTI-LEVEL LOGIC



LOGIC SYNTHESIS



- A node is represented as a Boolean network- A network of Boolean operators reflects the structure and function of the nodes.
- A Boolean network is defined as Directed Acyclic Graph (DAG).
- Node in DAG represents an arbitrary Boolean function
- Edge (i,j) in DAG represents data dependency between nodes i and j



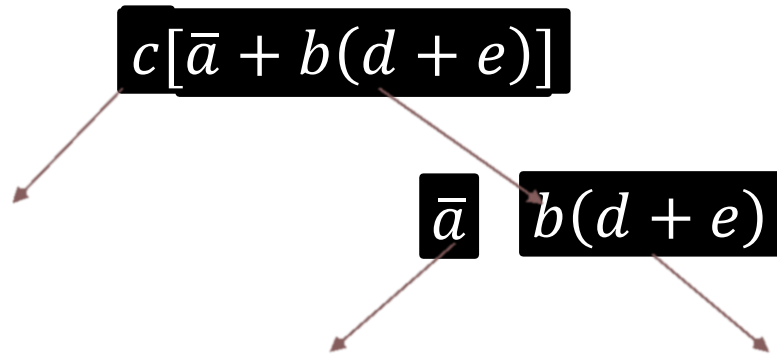
NODE REPRESENTATION

- Sum of products form
- Factored form
- Binary Decision Diagram



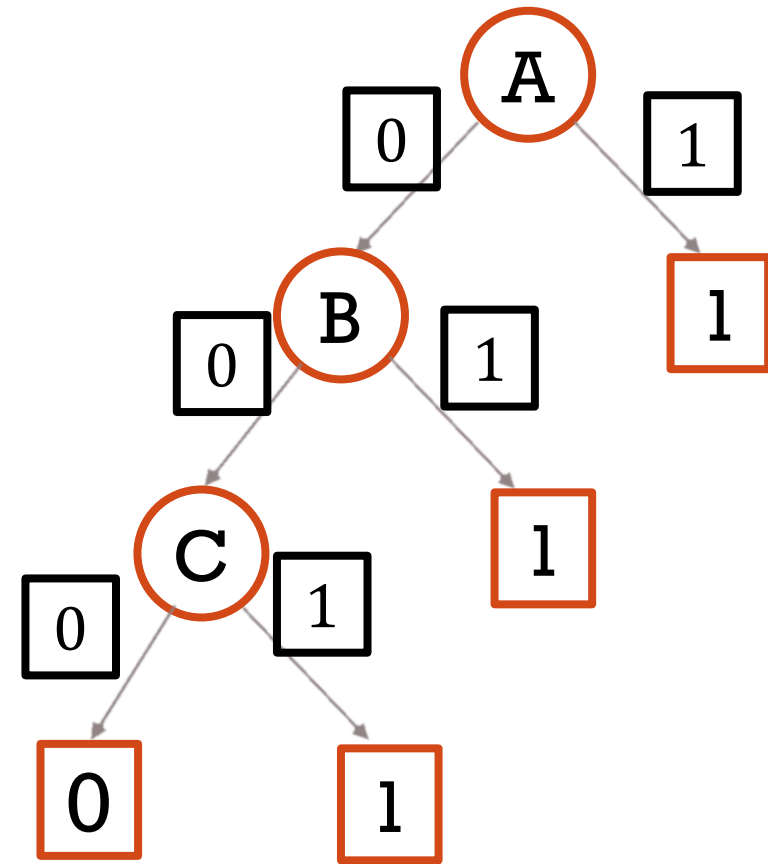
FACTORED FORM

- Defined recursively either as single literal or as a product or a sum of two factored forms

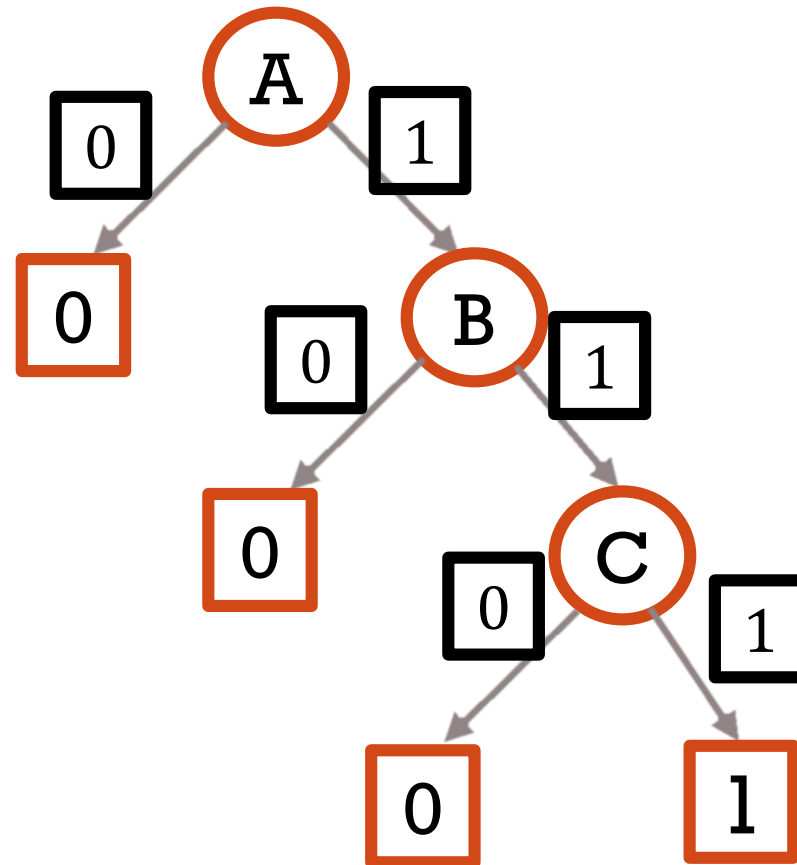


BINARY DECISION DIAGRAM

$$A + B + C$$



$$A \cdot B \cdot C$$



BINARY DECISION DIAGRAM

- A binary decision diagram (BDD) is a rooted Directed Acyclic Graph used to represent a Boolean function.
- Two kinds of nodes
 - Constant node
 - Variable Node



VARIABLE NODE

⑩ Variable node v (a non-terminal) is labeled by a Boolean variable $x_{\text{index}(v)}$ where $\text{index}(v) \in \{1, \dots, n\}$.

⑩ It has two children:

$\text{low}(v)$: the child reached when $x_{\text{index}(v)} = 0$ (false)

$\text{high}(v)$: the child reached when $x_{\text{index}(v)} = 1$ (true)



CONSTANT NODE

- Constant node v is terminal node
 $value(v) \in \{0, 1\}$



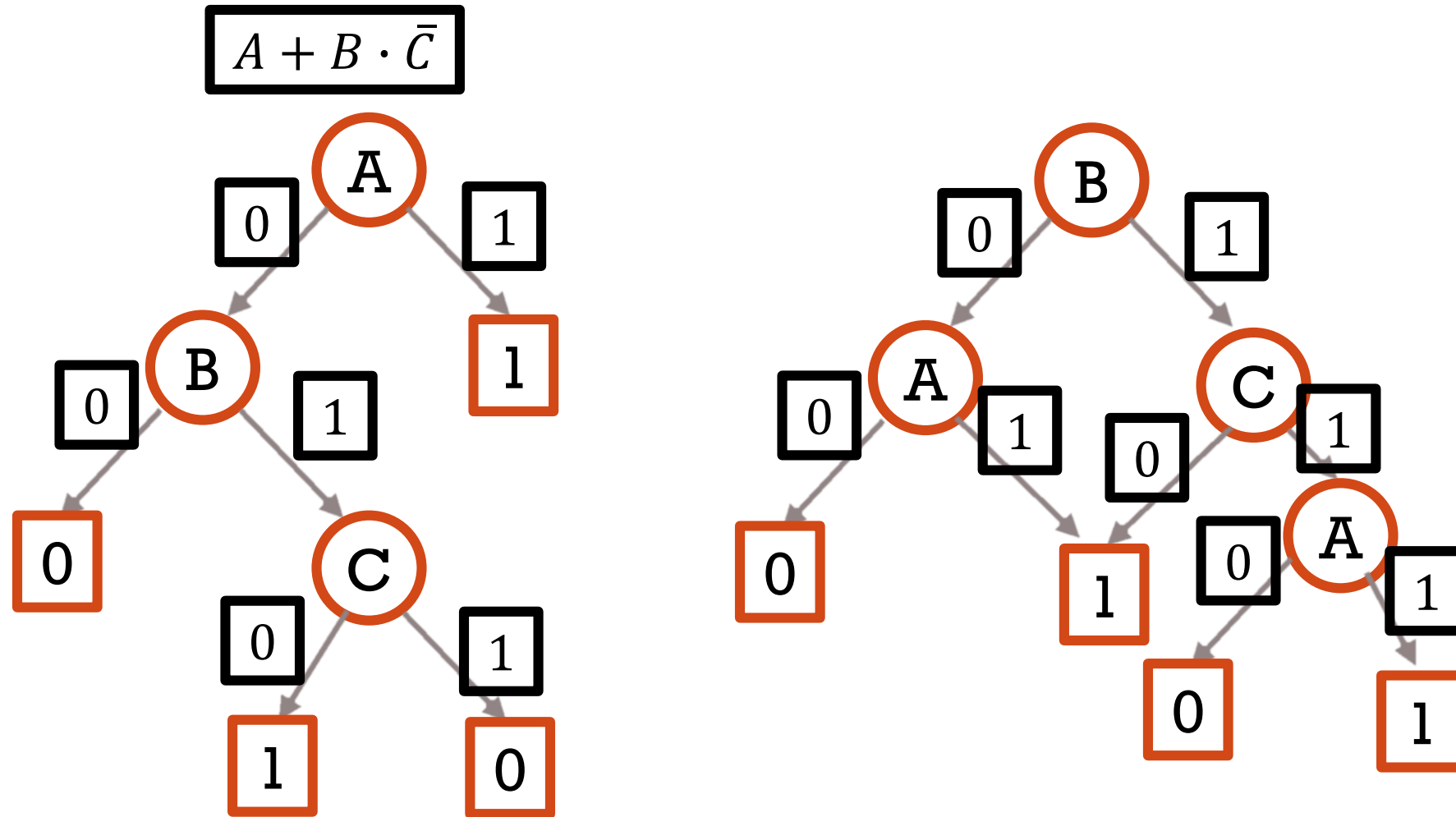
SHANNON EXPANSION THEOREM

Any Boolean function can be decomposed with respect to a variable into a multiplexer form.

$$f(x_1, x_2, \dots, x_n) = x_i \cdot f(x_i = 1) + \overline{x_i} \cdot f(x_i = 0)$$

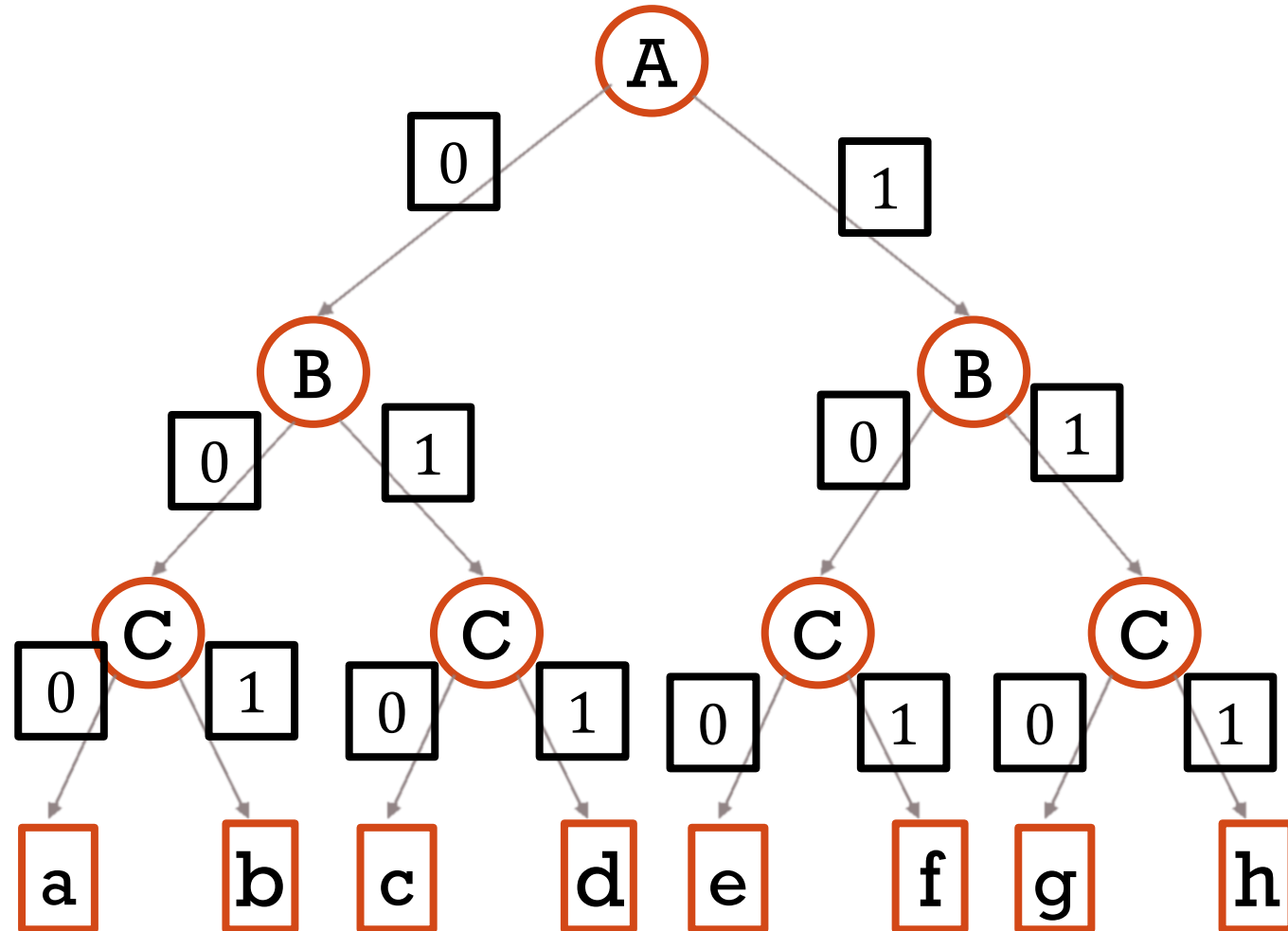


Binary Decision Diagrams



Complete BDD from truth table:

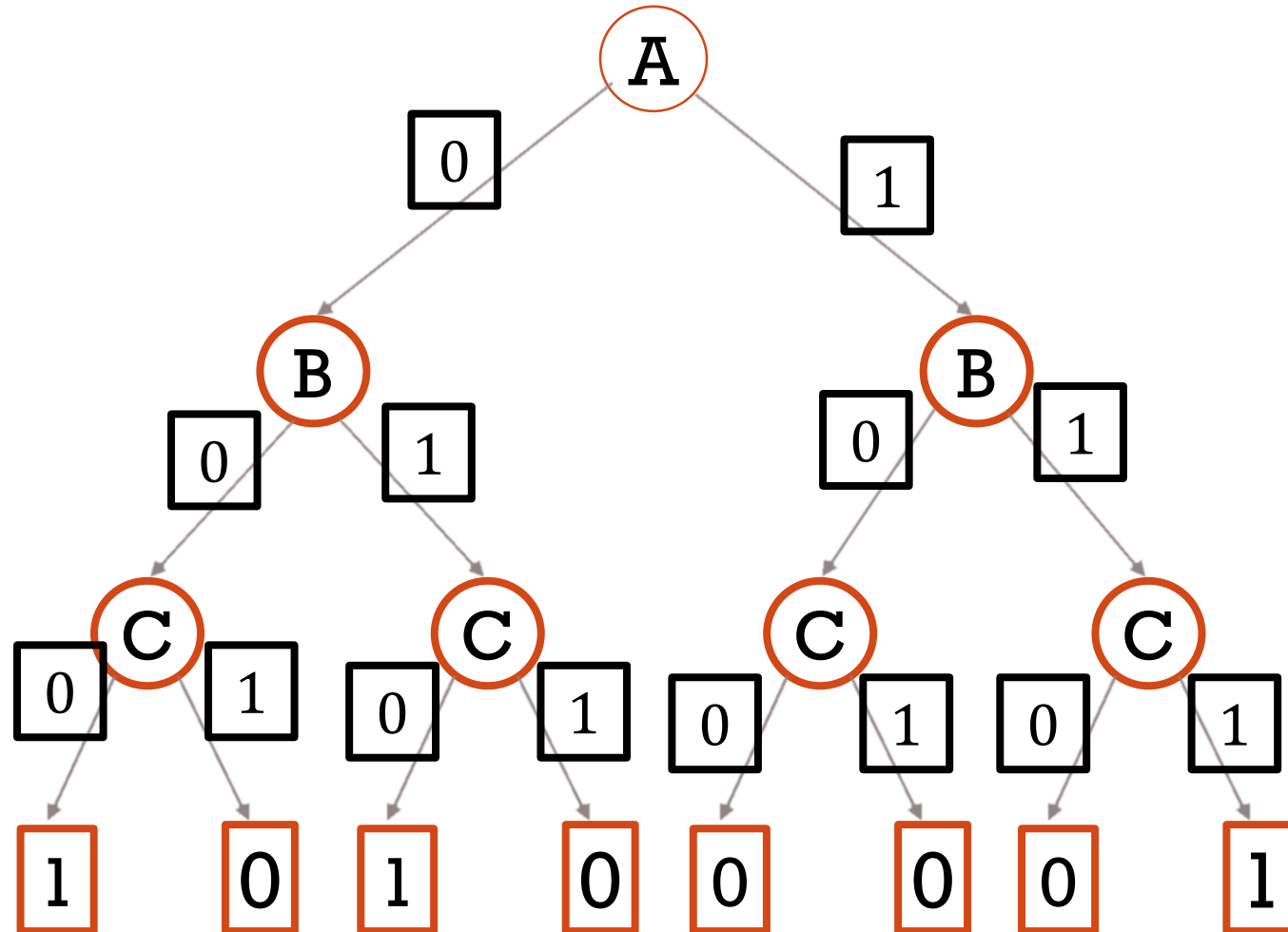
A	B	C	f
0	0	0	a
0	0	1	b
0	1	0	c
0	1	1	d
1	0	0	e
1	0	1	f
1	1	0	g
1	1	1	h



Binary Decision Diagrams

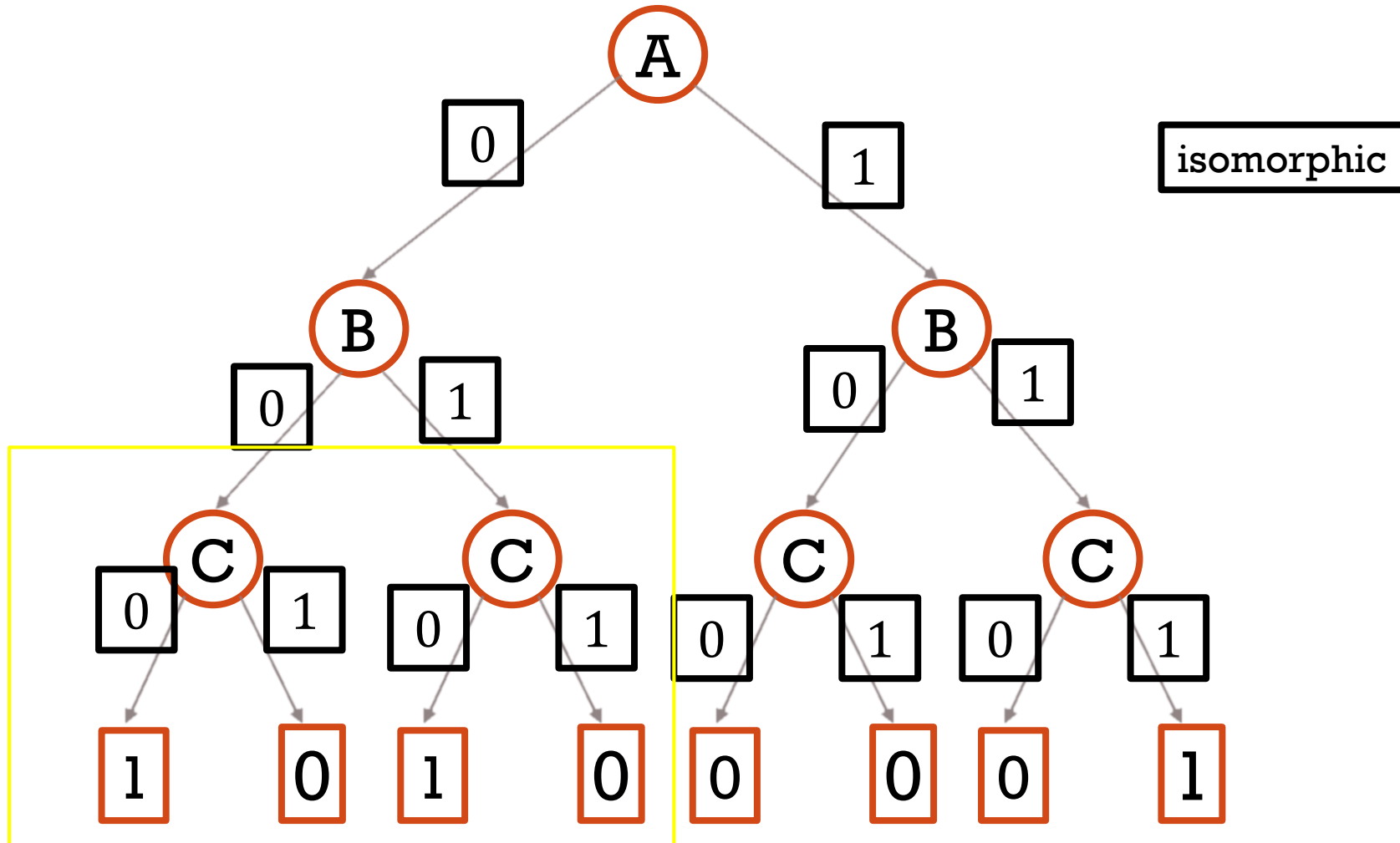
$$A \cdot B \cdot C + \bar{A} \cdot \bar{C}$$

isomorphic



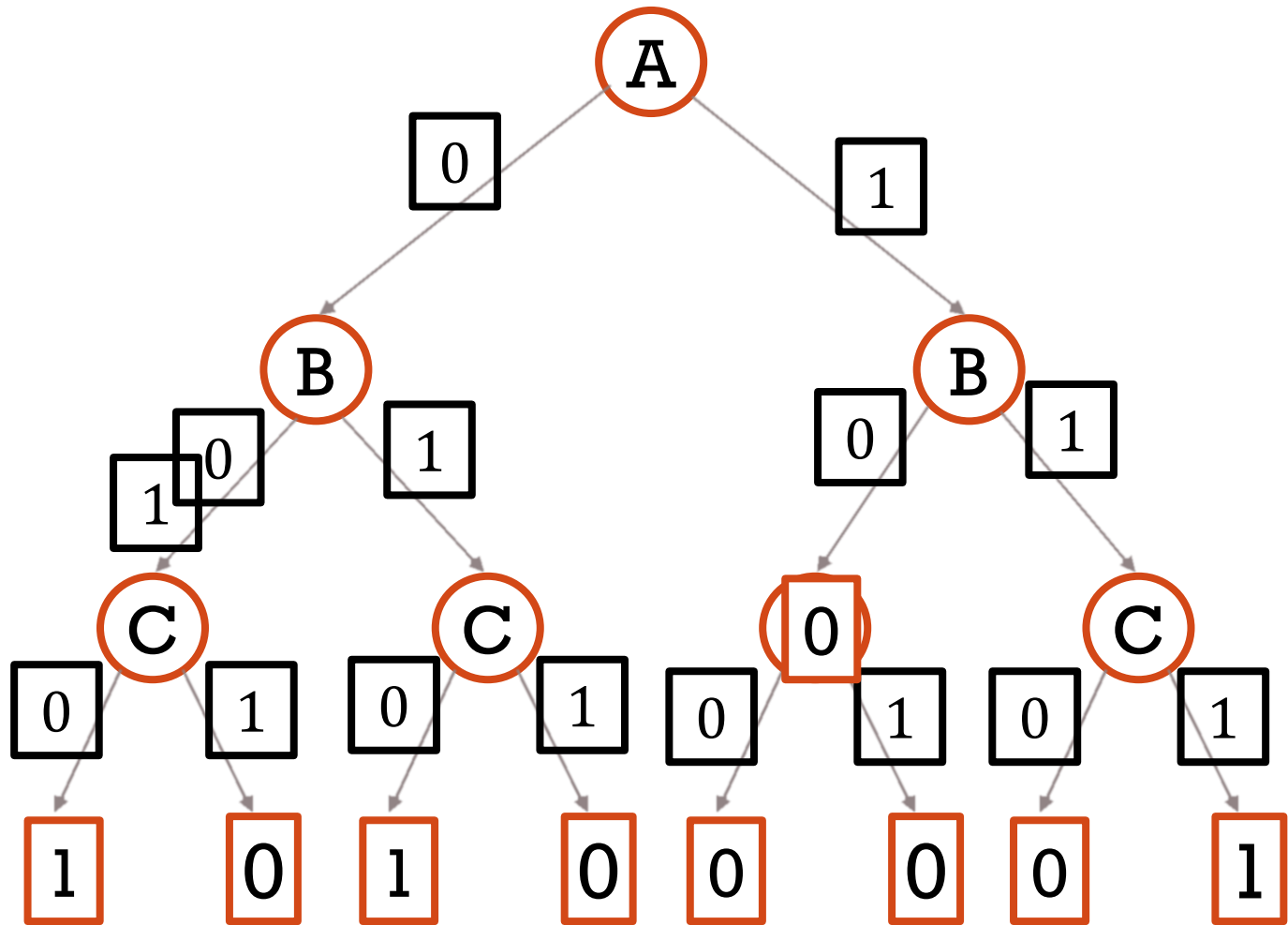
BDD Simplification

$$A \cdot B \cdot C + \bar{A} \cdot \bar{C}$$



BDD Simplification

$$A \cdot B \cdot C + \bar{A} \cdot \bar{C}$$



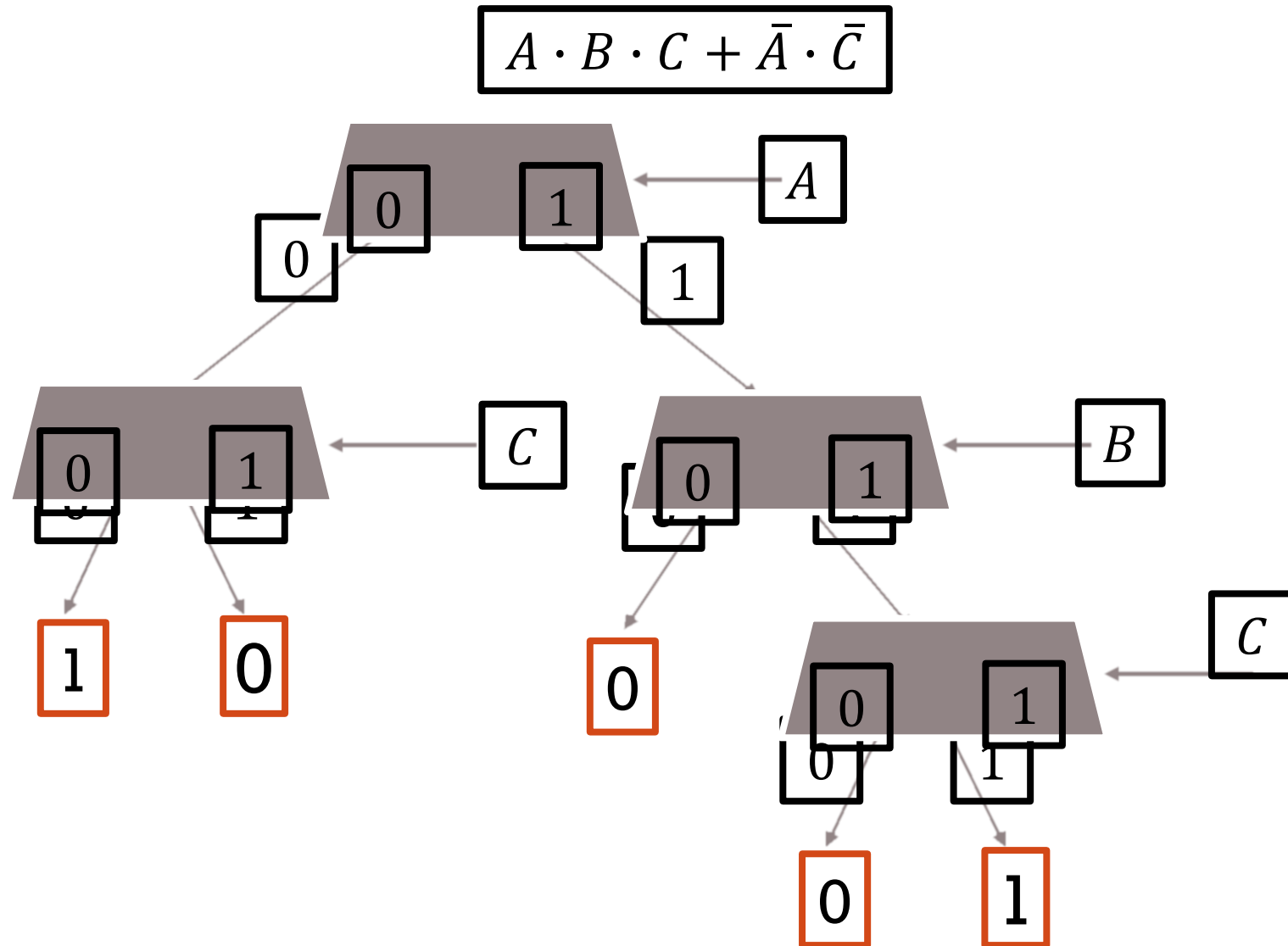
SHANNON EXPANSION THEOREM

Any Boolean function can be decomposed with respect to a variable into a multiplexer form.

$$f(x_1, x_2, \dots, x_n) = x_i \cdot f(x_i = 1) + \overline{x_i} \cdot f(x_i = 0)$$



Implementation by Mux



Binary Decision Diagrams

Ordered Binary Decision Diagram (OBDD)

A BDD in which an ordering relation among the nodes exists is called a ordered BDD (OBDD)

The non-terminal nodes are ordered from the root to the terminal nodes.



BDDs are not canonical (several BDD rep. for same Boolean function)

ROBDDs are canonical and compact

BDD $\rightarrow$ ROBDD

Fix order of variables

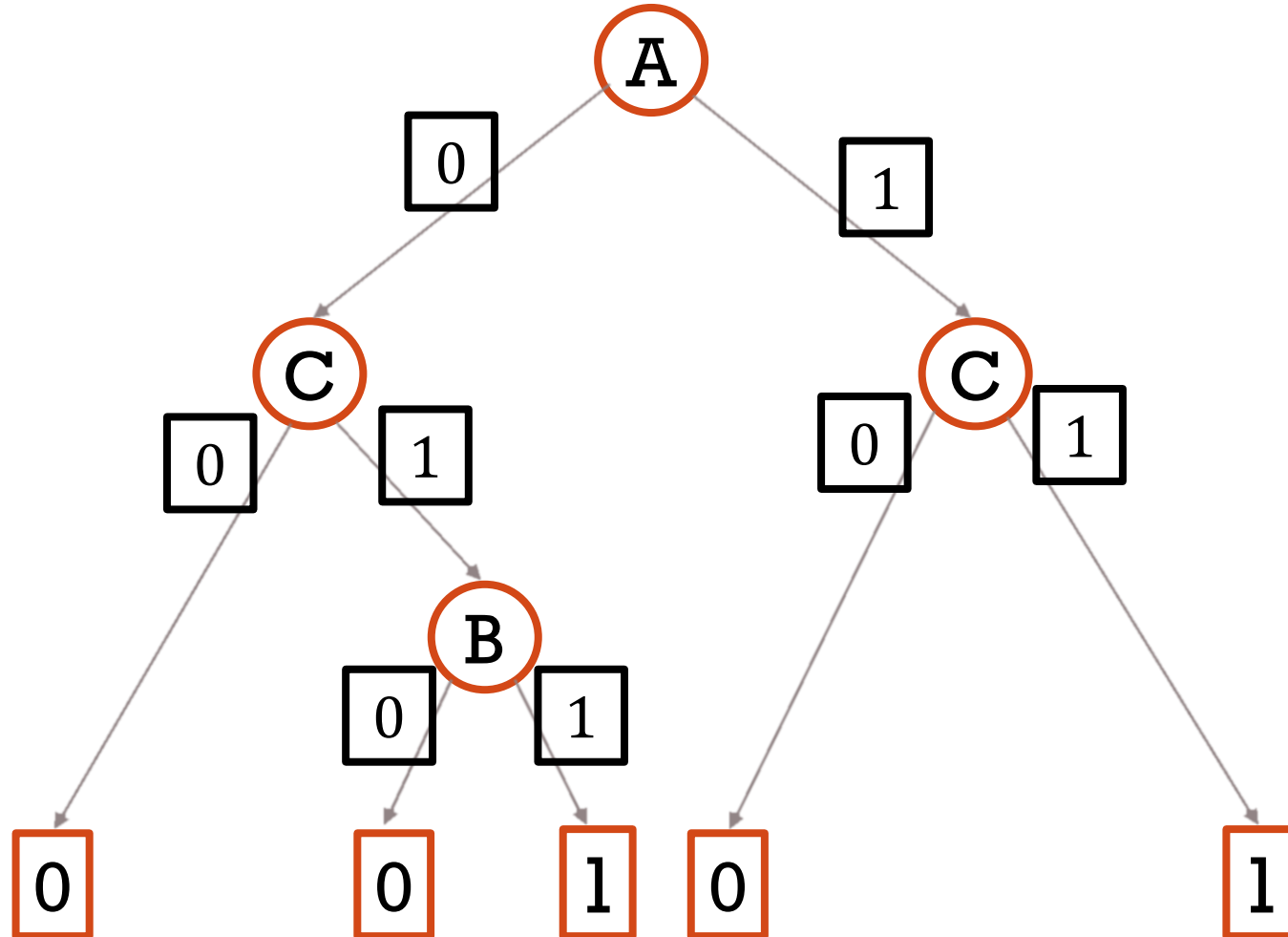
Delete Redundant nodes

Share subgraphs that represent the same function

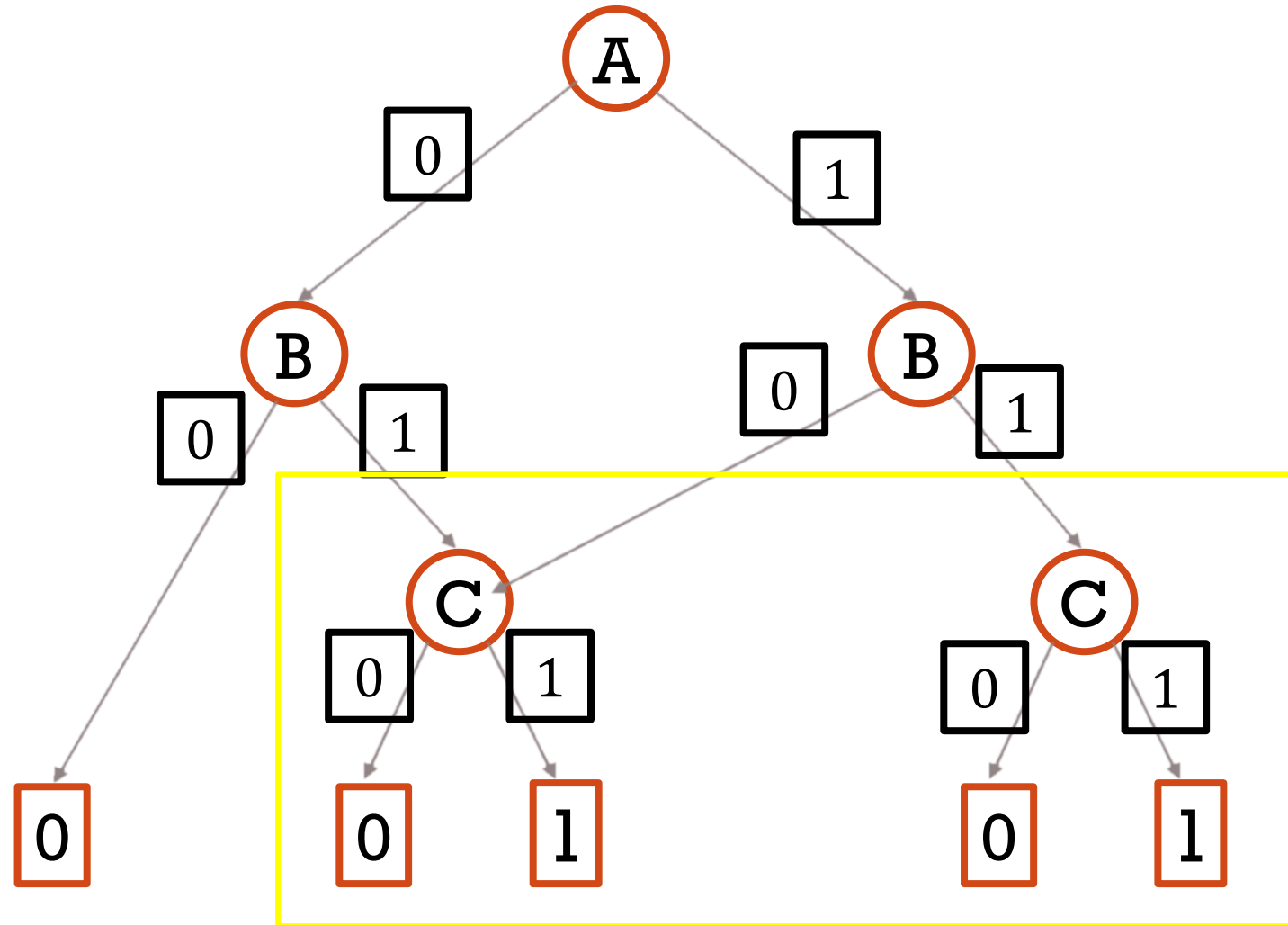


Binary Decision Diagrams

$(A + B) \cdot C$ with order A, C, B

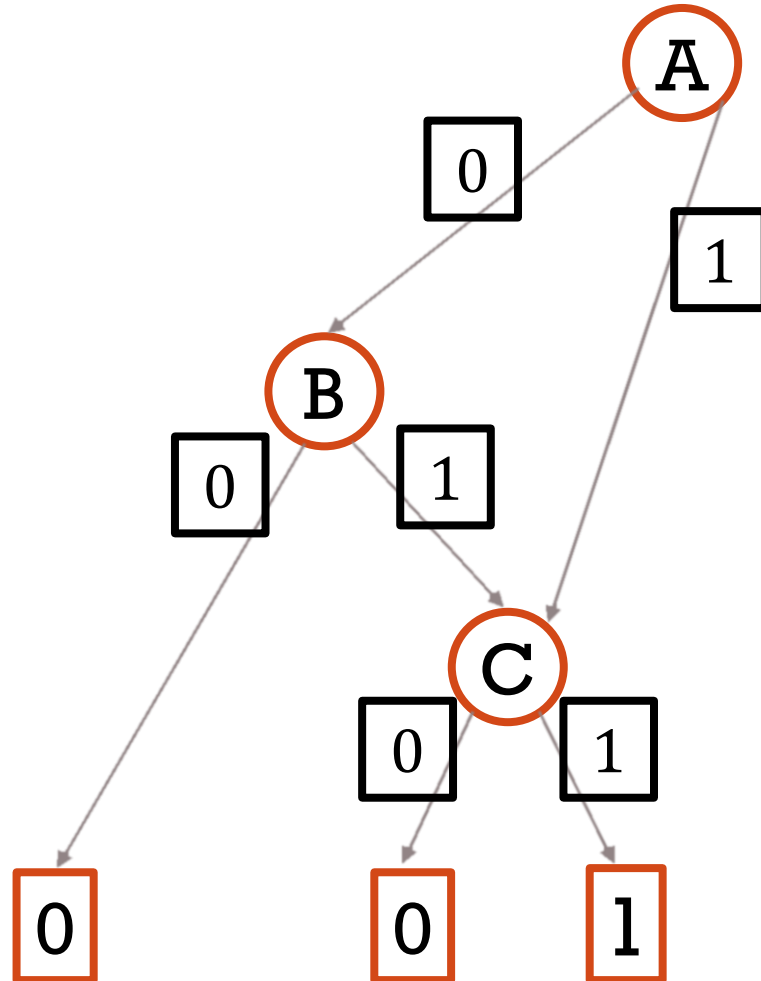


$(A + B) \cdot C$ with order A, B, C



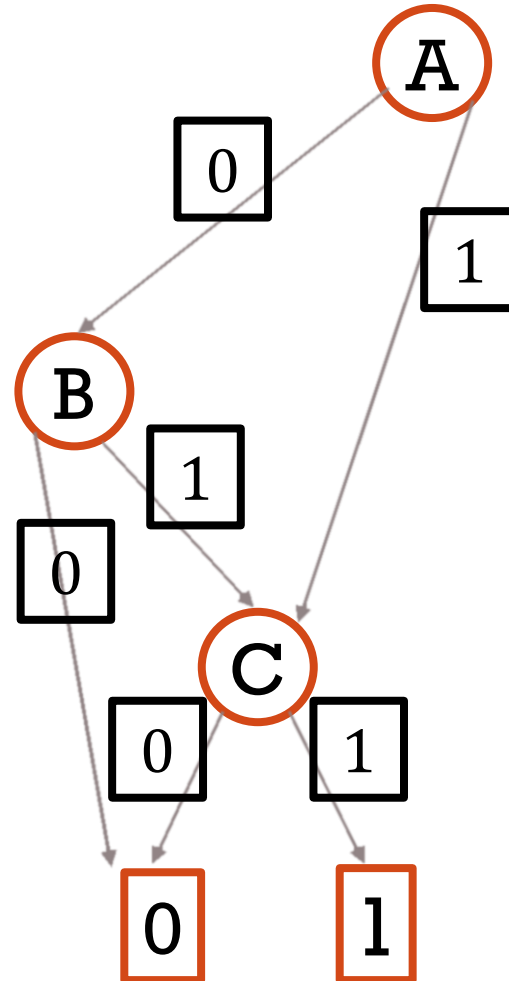
Binary Decision Diagrams

$(A + B) \cdot C$ with order A, B, C

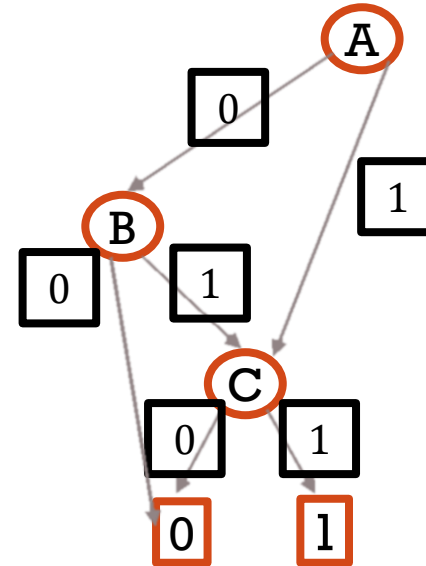
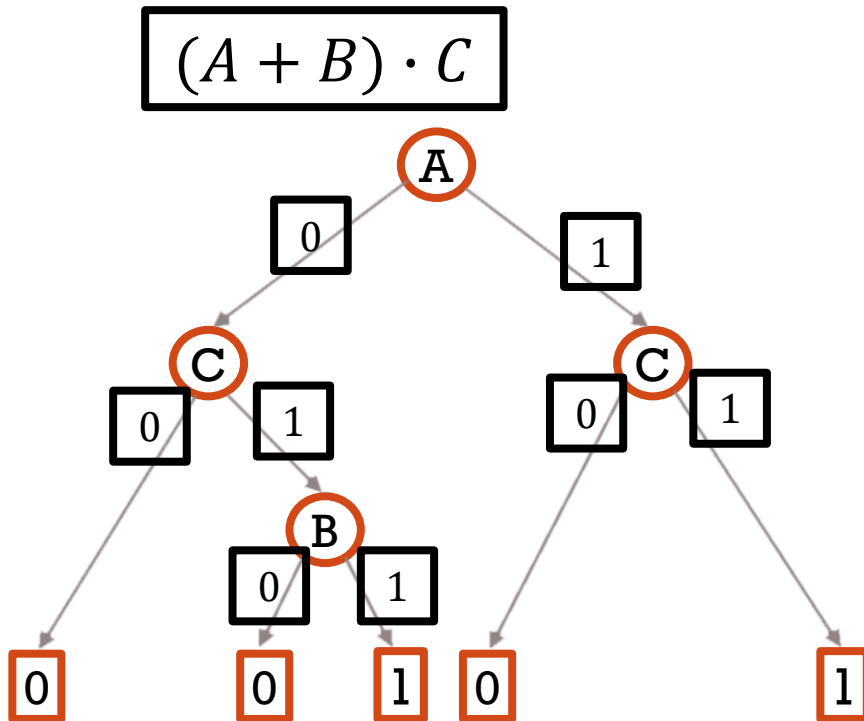


Binary Decision Diagrams

$(A + B) \cdot C$ with order A, B, C



Binary Decision Diagrams



Find OBDD with order C, B A

